

Dokumentacja implementacyjna - Wyznaczanie współrzędnych węzłów dla wizualnej reprezentacji grafu planarnego w języku C

 Aleksander Jóźwik, Anastasiya Kryvetskaya

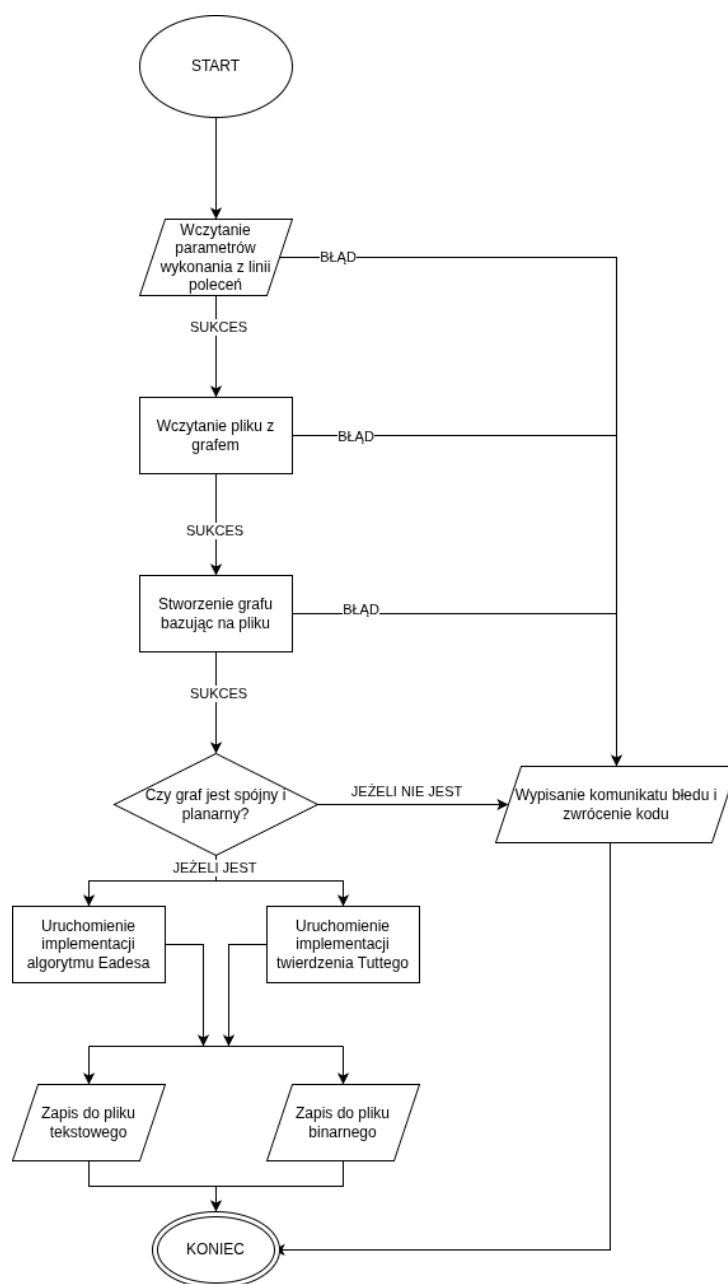
 16 kwietnia 2026

Spis treści

1	Schemat blokowy działania programu	3
2	Podział programu na moduły	4
3	Opis głównego pliku	5
4	Opis poszczególnych modułów	6
4.1	Moduł input	6
4.1.1	Funkcja create_params	6
4.1.2	Funkcja load_params	6
4.2	Moduł vector	6
4.2.1	Struktury	7
4.2.2	Funkcja add_vectors	7
4.2.3	Funkcja count_vector_length	7
4.2.4	Funkcja distance	7
4.2.5	Funkcja vector_from_points	7
4.2.6	Funkcja mult_by_num	8
4.2.7	Funkcja negative	8
4.2.8	Funkcja print_list	8
4.2.9	Funkcja subtract_vectors	8
4.3	Moduł validation	9
4.3.1	Funkcja is_graph_valid	9
4.4	Moduł output	9
4.4.1	Funkcja save_to_text	9
4.4.2	Funkcja save_to_bin	10
4.5	Moduł graph	11
4.5.1	Definicja struktur	11
4.5.2	Funkcja load_graph	11
4.5.3	Funkcja free_graph	12
4.5.4	Funkcja build_adj_list	12
4.5.5	Funkcja print_adj_list	13
4.5.6	Funkcja print_outer_face	13
4.5.7	Funkcja print_nodes_pos	13
4.5.8	Funkcja free_adj_list	14
4.5.9	Funkcja free_deg	14
4.5.10	Funkcja find_outer_face	14
4.5.11	Funkcja get_center	14
4.5.12	Funkcja place_on_circle	15
4.6	Moduł eades	15

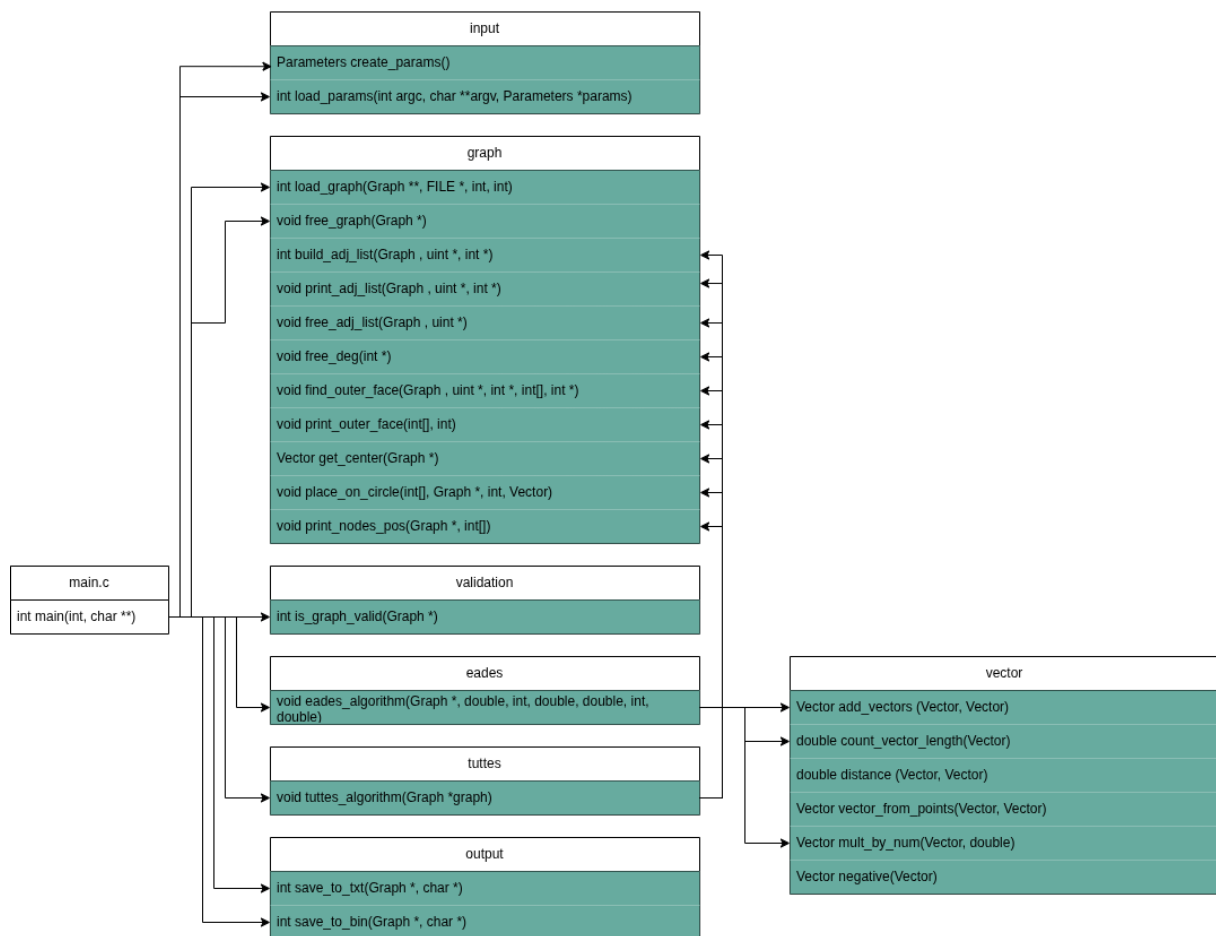
4.6.1	Funkcja eades_algorithm	15
4.7	Moduł toutes	16
4.7.1	Funkcja toutes_algorithm	16

1 Schemat blokowy działania programu



Rysunek 1: Diagram blokowy ilustrujący sposób działania programu

2 Podział programu na moduły



Rysunek 2: Diagram podziału programu na pliki

Program został podzielony na 6 modułów z których każdy składa się z pliku C i pliku nagłówkowego. Moduły to:

- input - odpowiada za wczytywanie parametrów z linii poleceń.
- graph - odpowiada za zdefiniowanie struktur dla grafu, tworzenie grafu bazując na wczytanym pliku, zarządzanie pamięcią zaalokowaną dla struktur grafu.
- validation - odpowiada za sprawdzenie czy graf jest spójny i planarny.
- eades - odpowiada za uruchomienie funkcji implementującej algorytm Eadesa.
- tuttes - odpowiada za uruchomienie funkcji implementującej twierdzenie Tuttego.
- vector - odpowiada za wykonywanie operacji na wektorach.
- output - odpowiada za zapisywanie informacji o pozycji wierzchołków do pliku tekstowego lub binarnego.

3 Opis głównego pliku

Plik `main.c` zarządza przebiegiem programu oraz odpowiada za wczytywanie parametrów wykonania programu z linii poleceń.

```
1 int main(int argc, char **argv) {
2     // Pobranie parametrów wykonania programu z linii poleceń
3     Parameters params = create_params();
4     Jeśli load_params(argc,argv,&params) nie działa:
5         return EXIT_FAILURE
6
7     Jeśli nie udało się otworzyć
8         return ERR_FILE_OPEN;
9
10
11     // Sprawdzenie czy ziarno zostało podane jako argument linii poleceń
12     if(params.wasSeedProvided == 1)
13         srand(params.seed);
14     else
15         srand(time(NULL));
16
17     // Stworzenie grafu na podstawie pliku
18     int load_graph_status = 0;
19     Graph *graph = load_graph(params.graph_file, params.width, params.height, &load_graph_status);
20
21     Jeśli nie udało się wczytać graf
22         return ERR_GRAPH_LOAD;
23
24     Jeśli nie udało się alokacja pamięci
25         return ERR_MEMORY_ALLOC;
26 }
27
28 Sprawdzenie czy graf jest spójny, planarny, i alokację pamięci,
29 jeśli nie, to zwracanie odpowiedniego kodu błędu
30
31 // Obsługa wyboru algorytmu z linii poleceń
32 Jeśli wybrany Eades
33     eades_algorithm(graph, params.minimum_force, params.max_iterations, params.ideal_len,
34         ↪ params.spring_const,
35         ↪ params.repulsion_const, params.cooling);
36
37 Jeśli wybrany Tutes
38     tutes_algorithm(graph, params.max_iterations);
39
40 Domyślnie
41     eades_algorithm(graph, params.minimum_force, params.max_iterations, params.ideal_len,
42         ↪ params.spring_const, params.repulsion_const, params.cooling);
43
44 // Obsługa zapisu do wybranego typu pliku
45 Jeśli nie udało się zapisać do pliku tekstowego
46     return ERR_SAVE_TEXT_FAILED;
47 Jeśli nie udało się zapisać do pliku binarnego
48     return ERR_SAVE_BINARY_FAILED;
49
50 free(params.output_name);
51 fclose(params.graph_file);
52
53 printf("Pomyślnie zapisano informacje o %d wierzchołkach do pliku.\n", graph->num_nodes);
54 free_graph(graph);
55 return EXIT_SUCCESS;
56 }
```

Na koniec program zwalnia pamięć, zamyka pliki i wyświetla komunikat o pomyślnym zapisaniu danych.

4 Opis poszczególnych modułów

4.1 Moduł input

Moduł ten służy do wczytania parametrów linii poleceń do struktury, które będzie je przechowywać.

```
1 typedef struct {
2     int algorithm;
3     int output_type;
4     char *output_name;
5     FILE *graph_file;
6
7     int seed;
8     int wasSeedProvided;
9     int width;
10    int height;
11
12    // Parametry algorytmu Eadesa z wartościami domyślnymi
13    double minimum_force;
14    int max_iterations;
15    double ideal_len;
16    double repulsion_const;
17    double spring_const;
18    double cooling;
19 } Parameters;
```

Moduł definiuje strukturę **Parameters**, która służy do przechowywania parametrów.

4.1.1 Funkcja create_params

Tworzy nową strukturę i ustawia wartości zmiennych na domyślne. Funkcja zwraca strukturę **Parameters**.

4.1.2 Funkcja load_params

Funkcja pobiera parametry podane z linii poleceń (udostępnione z funkcji `main`) i umieszcza je w strukturze **Parameters**.

Argumenty:

- `argc` - ilość parametrów
- `argv` - tablica parametrów
- `params` - wskaźnik na strukturę **Parameters**

Funkcja zwraca `EXIT_SUCCESS` dla sukcesu i `ERR_MISSING` jeżeli użyto nieznanej opcji.

4.2 Moduł vector

Moduł udostępnia funkcję służącą do wykonywania operacji na wektorach, a także definiuje ich strukturę. Funkcje tego modułu wykorzystywane są przez moduł `eades` i `tuttes`.

4.2.1 Struktury

```
1 typedef struct {  
2     double x;  
3     double y;  
4 } Vector;
```

Struktura wektora składa się dwóch liczb zmiennoprzecinkowych dla współrzędnych x i y.

4.2.2 Funkcja add_vectors

Funkcja dodaje do siebie dwa wektory.

Argumenty:

- a - pierwszy wektor
- b - drugi wektor

Funkcja zwraca zmieniony pierwszy wektor.

4.2.3 Funkcja count_vector_length

Funkcja oblicza długość wektora.

Argumenty:

- a - wektor

Funkcja zwraca długość wektora jako wartość typu double.

4.2.4 Funkcja distance

Funkcja oblicza odległość między dwoma wektorami zgodnie z formułą $d = \sqrt{(x_b - x_a)^2 + (y_b - y_a)^2}$

Argumenty:

- a - pierwszy wektor
- b - drugi wektor

Funkcja zwraca odległość między wektorami jako wartość typu double.

4.2.5 Funkcja vector_from_points

Funkcja tworzy wektor przemieszczenia od punktu reprezentowanego przez wektor a do punktu reprezentowanego przez wektor b.

Argumenty:

- a - pierwszy wektor
- b - drugi wektor

Funkcja zwraca nowy wektor.

4.2.6 Funkcja `mult_by_num`

Funkcja mnoży wektor przez skalar.

Argumenty:

- `a` - wektor
- `n` - wartość liczbowa typu `double`

Funkcja zwraca nowy wektor, będący wynikiem mnożenia wektora `a` przez skalar `n`.

4.2.7 Funkcja `negative`

Funkcja zmienia znak współrzędnych wektora na przeciwny.

Argumenty:

- `a` - wektor

Funkcja zwraca nowy wektor.

4.2.8 Funkcja `print_list`

Wypisuje zawartość tablicy liczb całkowitych na standardowe wyjście. Każdy element jest prezentowany wraz z odpowiadającym mu indeksem w formacie `[indeks] - wartość`.

Argumenty:

- `list` - tablica liczb całkowitych .
- `size` - liczba elementów znajdujących się w tablicy.

4.2.9 Funkcja `subtract_vectors`

Funkcja odejmowania dwóch wektorów dwuwymiarowych. Oblicza różnicę współrzędnych `x` oraz `y`.

Argumenty:

- `a` - wektor od którego odejmujemy.
- `b` - wektor który odejmujemy.

Funkcja zwraca nową strukturę `Vector` będącą wynikiem operacji `a-b`.

4.3 Moduł validation

Moduł ten odpowiada za sprawdzenie grafu pod kątem planarności i spójności po jego wczytaniu z pliku. Moduł składa się z 4 funkcji, które zostaną opisane poniżej.

4.3.1 Funkcja is_graph_valid

```
1 int is_graph_valid(Graph *graph){
2     if(is_potentially_planar(graph) == 0)
3         return ERR_GRAPH_NOT_PLANAR;
4     else if(is_graph_connected(graph) == 0){
5         return ERR_GRAPH_NOT_CONNECTED;
6     }
7     else if(is_graph_planar(graph) == 0){
8         return ERR_GRAPH_NOT_PLANAR;
9     }
10    return 0;
11 }
```

Jest to funkcja publiczna używana przez plik main.c, która używa innych funkcji tego modułu w celu weryfikacji grafu i w razie potrzeby zwrócenia odpowiednich kodów błędu.

Argumenty:

- graph - wskaźnik na strukturę grafu.

Funkcja zwraca liczbę całkowitą int informującą o rezultacie funkcji.

4.4 Moduł output

Moduł ten odpowiada za wypisywanie danych do pliku tekstowego lub binarnego.

4.4.1 Funkcja save_to_text

```
1 save_to_txt(graf, nazwa_wyjściowa):
2
3     Jeśli graf nie istnieje -> zwróć błąd (1).
4
5     Ustal nazwę pliku:
6
7         Jeśli podano nazwa_wyjściowa, użyj jej.
8
9         W przeciwnym razie użyj nazwy domyślnej (OUTPUT_NAME_TXT).
10
11    Otwórz plik do zapisu:
12
13        Jeśli nie udało się otworzyć pliku -> zwróć błąd (1).
14
15    Dla każdego wierzchołka w grafie:
16
17        Zapisz do pliku w jednej linii: identyfikator, pozycja_x, pozycja_y.
18
19    Zamknij plik.
20
21    Zwróć sukces (0).
22
23
```

Funkcja ta odpowiada za wypisywanie informacji o wierzchołkach grafu do pliku tekstowego o wybranej nazwie. Jeżeli nazwa nie została podana to zostaje ona ustawiona na domyślną ("output.txt"). Plik tekstowy jest zgodny z formatem ustalony w dokumentacji funkcjonalnej.

```
1 <numer_wierzchołka> <współrzędna_x> <współrzędna_y>
```

Argumenty:

- graph - wskaźnika na strukturę grafu
- output_name - nazwa pliku wyjściowego

Funkcja zwraca liczbę całkowitą równą 1 jeżeli operacja się niepowiodła i 0 jeżeli się powiodła.

4.4.2 Funkcja save_to_bin

```
1 int save_to_bin(Graph *graph, char *output_name) {
2     if(!graph) return 1;
3
4     char *final_name = output_name ? output_name : OUTPUT_NAME_BIN;
5
6     FILE *file = fopen(final_name, "wb");
7     if(!file) return 1;
8
9     uint32_t num_nodes = (uint32_t) graph->num_nodes;
10    fwrite(&num_nodes, sizeof(uint32_t), 1, file);
11    for(int i = 0; i < graph->num_nodes; i++) {
12        uint32_t index = (uint32_t) graph->nodes[i].id;
13        float x = (float) graph->nodes[i].position.x;
14        float y = (float) graph->nodes[i].position.y;
15
16        fwrite(&index, sizeof(uint32_t), 1, file);
17        fwrite(&x, sizeof(float), 1, file);
18        fwrite(&y, sizeof(float), 1, file);
19    }
20    fclose(file);
21    return 0;
22 }
```

Funkcja save_to_bin służy do wypisywania danych do pliku binarnego. Podobnie jak dla poprzedniej funkcji, jeżeli nazwa pliku nie została podana to jest ona ustawiona na domyślną ("output.bin"). Dane są zapisywane w kolejności i typie zdefiniowanym w dokumentacji funkcjonalnej.

Argumenty:

- graph - wskaźnika na strukturę grafu
- output_name - nazwa pliku wyjściowego

Funkcja zwraca liczbę całkowitą równą 1 jeżeli operacja się niepowiodła i 0 jeżeli się powiodła.

4.5 Moduł graph

Moduł ten służy do tworzenia i zarządzania grafami.

4.5.1 Definicja struktur

```
1 typedef struct {
2     int u, v;
3     double weight;
4     char *name;
5 } Edge;
6
7 typedef struct {
8     Vector position;
9     Vector force;
10    uint id;
11 } Node;
```

Zdefiniowane są struktury dla wierzchołków oraz krawędzi grafu. Należy zwrócić szczególną uwagę na fakt, że pole `id` w wierzchołku nie jest używane w dalszej części programu oprócz w czasie wypisywania do pliku. Jest ono traktowane tylko jako etykieta. Aby operować na wierzchołkach program stosuje indeks tabeli `nodes` struktury grafu.

```
1 typedef struct{
2     Node *nodes;
3     int num_nodes;
4     int capacity_nodes;
5
6     Edge *edges;
7     int num_edges;
8     int capacity_edges;
9     int width;
10    int height;
11 } Graph;
```

Zdefiniowano również strukturę grafu. Która zawiera tablice dynamiczne wierzchołków i krawędzi wraz z ich ilością elementów oraz pojemnością. Struktura zawiera również szerokość i wysokość obszaru, w którym wyświetlony będzie graf.

4.5.2 Funkcja `load_graph`

Funkcja ta tworzy graf na podstawie wczytanego pliku.

Argumenty:

- `graph_file` - wskaźnik na plik
- `width` - szerokość obszaru, w którym wyświetlony będzie graf
- `height` - wysokość obszaru, w którym wyświetlony będzie graf
- `out_code` - kod błędu

Funkcja zwraca wskaźnik na strukturę grafu.

```
1
2 graph load_graph(plik, szerokość, wysokość, out_kod):
3     1. JEŚLI plik nie istnieje -> ustaw błąd i zwróć NULL.
4
5     2. ALOKUJ pamięć dla struktury Graph:
6         - Ustaw początkową pojemność (16) dla wierzchołków i krawędzi.
```

```

7      - Zainicjalizuj wymiary obszaru wyświetlania.
8      - JEŚLI błąd alokacji -> zwróć NULL.
9
10     3. DOPÓKI można odczytać dane z pliku (identyfikator, u, v, waga):
11     - Szukaj wierzchołka 'u' w grafie:
12     - Jeśli nie istnieje -> dodaj go do tablicy.
13     - Szukaj wierzchołka 'v' w grafie:
14     - Jeśli nie istnieje -> dodaj go do tablicy.
15     - DODAJ krawędź między 'u' a 'v' z podaną wagą i nazwą.
16
17     - JEŚLI podczas dodawania zabraknie pamięci -> zwolnij wszystko i zwróć NULL.
18
19     4. ZWRÓĆ gotowy wskaźnik na graf.

```

4.5.3 Funkcja free_graph

Funkcja służy do zwalniania pamięci zaalokowanej dla struktury grafu.

Argumenty:

- graph - wskaźnik na graf

```

1 void free_graph(Graph *graph) {
2     if (graph) {
3         for (int i = 0; i < graph->num_edges; i++) {
4             free(graph->edges[i].name);
5         }
6
7         free(graph->edges);
8         free(graph->nodes);
9         free(graph);
10    }
11 }

```

Funkcja dealokuje nazwy krawędzi oraz tablice krawędzi i wierzchołków oraz samą strukturę grafu. Oto fragment dokumentacji implementacyjnej przygotowany na podstawie dostarczonego kodu, z zachowaniem Twojej składni LaTeX oraz oryginalnych komentarzy.

4.5.4 Funkcja build_adj_list

```

1 int build_adj_list(Graph *graph, uint **adj_list, int *deg) {
2     int n_nodes = graph->num_nodes;
3     int n_edges = graph->num_edges;
4
5     // Zapełnienie deg
6     for (int i = 0; i < n_nodes; i++)
7         deg[i] = 0;
8
9     // Zapełnienie adj_list
10    for (int i = 0; i < n_edges; i++) {
11        int u = graph->edges[i].u;
12        int v = graph->edges[i].v;
13
14        adj_list[u][deg[u]++] = v;
15        adj_list[v][deg[v]++] = u;
16    }
17    return EXIT_SUCCESS;
18 }

```

Funkcja ta przekształca listową reprezentację krawędzi w listę sąsiedztwa, co jest używane przez algorytm tuttesa dla efektywnego poszukiwania sąsiadów. W pierwszym kroku zeruje tablicę stopni wierzchołków, a następnie iteruje po krawędziach, dodając sąsiadów do odpowiednich list.

Argumenty:

- graph - wskaźnik na strukturę grafu.
- adj_list - tablica dwuwymiarowa przechowująca sąsiadów wierzchołków.
- deg - tablica przechowująca liczbę sąsiadów dla każdego wierzchołka.

Funkcja zwraca EXIT_SUCCESS po poprawnym zbudowaniu struktury.

4.5.5 Funkcja print_adj_list

(*)Funkcja służy wyłącznie dla dyagnostyki. Jej wywołanie jest zakomentowane w kodzie.

Wypisuje kompletną listę sąsiedztwa na standardowe wyjście. **Argumenty:**

- graph - wskaźnik na strukturę grafu.
- adj_list - tablica przechowująca sąsiadów.
- deg - tablica stopni wierzchołków.

Przykład wyjścia:

```
1 Sąsiedzi dla 0: 1
2 Sąsiedzi dla 1: 0 2 3
3 Sąsiedzi dla 2: 1 3
4 Sąsiedzi dla 3: 2 1
```

4.5.6 Funkcja print_outer_face

(*)Funkcja służy wyłącznie dla dyagnostyki. Jej wywołanie jest zakomentowane w kodzie.

Wizualizuje cykl tworzący zewnętrzny poligon grafu (outer face) w formacie czytelnej ścieżki. Pierwszy i ostatni element zawsze będą takie same, ponieważ jest to prezentacja zamkniętego cyklu

Argumenty:

- dfs_res - tablica przechowująca indeksy wierzchołków poligonu zewnętrznego.
- dfs_res_size - liczba elementów w tablicy.

Przykład wyjścia:

```
1 3->2->1->3
```

4.5.7 Funkcja print_nodes_pos

(*)Funkcja służy wyłącznie dla dyagnostyki. Jej wywołanie jest zakomentowane w kodzie.

Wypisuje współrzędne wszystkich wierzchołków, oznaczając te, które zostały przypisane do poligonu zewnętrznego.

Argumenty:

- graph - wskaźnik na strukturę grafu.
- is_fixed - tablica flag logicznych (1 - wierzchołek stały(z poligonu zewnętrznego), 0 - ruchomy).

Przykład wyjścia:

```

1 [0] - (30.000,72.000)
2 [1] - (55.000,48.000) -poligon zewnetrzny
3 [2] - (100.000,72.000) -poligon zewnetrzny
4 [3] - (36.000,28.000) -poligon zewnetrzny
5 =====

```

4.5.8 Funkcja free_adj_list

Funkcja służy do zwalniania pamięci zaalokowanej dla struktury listy sąsiedztwa.

Argumenty:

- graph - wskaźnik na strukturę grafu (używany do określenia liczby wierzchołków).
- adj_list - tablica sąsiadów do usunięcia.

4.5.9 Funkcja free_deg

Funkcja służy do zwalniania pamięci zaalokowanej dla struktury ilości sąsiadów.

Argumenty:

- deg - wskaźnik na tablicę stopni wierzchołków.

4.5.10 Funkcja find_outer_face

Funkcja ta stanowi punkt wejścia do algorytmu wyznaczania zewnętrznego poligonu. Jej zadaniem jest przygotowanie struktur pomocniczych (tablicy odwiedzin oraz tablicy przodków) i uruchomienie dedykowanego algorytmu przeszukiwania w głąb (dfs_rec_face), który zidentyfikuje pierwszy napotkany cykl.

Argumenty:

- graph - wskaźnik na strukturę grafu.
- adj_list - dynamiczna tablica dwuwymiarowa (lista sąsiedztwa).
- deg - tablica przechowująca stopnie wierzchołków.
- cycle_res - tablica, do której zostaną zapisane indeksy wierzchołków tworzących cykl.
- cycle_idx - wskaźnik na zmienną przechowującą liczbę znalezionych wierzchołków w cyklu.

4.5.11 Funkcja get_center

Funkcja oblicza współrzędne środka obszaru, na którym rysowany jest graf. Wynik jest wyznaczany na podstawie wymiarów (width i height) zapisanych w strukturze grafu.

Argumenty:

- graph - wskaźnik na strukturę grafu zawierającą wymiary obszaru roboczego.

Funkcja zwraca strukturę Vector zawierającą współrzędne punktu środkowego.

4.5.12 Funkcja place_on_circle

```
1 void place_on_circle(int outer_faces[], Graph *graph, int k, Vector center) {
2     // margin to odległość krańca okręga od granicy przestrzeni
3     int margin = 20;
4     // radius- promień domyślnego okręga na którym rozkładamy wierzchołki
5     double radius = fmin(graph->width, graph->height) / 2.0 - margin;
6
7     Po wierzchołkach zewnętrznego poligonu
8     liczymy kąt(patrz formułę poniżej)
9
10
11     Zmieniamy pozycje wierzchołków
12
13
14 }
```

Funkcja odpowiada za geometryczne rozmieszczenie wierzchołków wyznaczonych jako „poligon zewnętrzny” na okręgu. Jest to kluczowy etap przygotowawczy dla algorytmu Tutte’a, ponieważ unieruchomienie tych wierzchołków w formie wypukłego wielokąta gwarantuje poprawne rozmieszczenie pozostałych punktów wewnątrz. Formuła dla kątu :

$$\alpha = \frac{2\pi \cdot i}{k}$$

Argumenty:

- outer_faces - tablica indeksów wierzchołków do rozmieszczenia.
- graph - wskaźnik na strukturę grafu, w której zostaną zaktualizowane pozycje.
- k - liczba wierzchołków w wielokącie zewnętrznym.
- center - wektor współrzędnych środka, wokół którego budowany jest okrąg.

4.6 Moduł eades

Moduł ten zawiera implementację algorytmu Eadesa, należącego do grupy algorytmów siłowych (*force-directed graph drawing*). Służy on do automatycznego rozmieszczania wierzchołków grafu w przestrzeni dwuwymiarowej tak, aby krawędzie miały zbliżoną długość, a wierzchołki nie nakładały się na siebie.

4.6.1 Funkcja eades_algorithm

```
1 void eades_algorithm(Graph *graph,
2     double minimum_force, int max_iterations,
3     double ideal_len, double spring_const, int c, double cooling)
4 {
5     //Inicjalizacja zmiennych
6     int t = 0;
7     double max_force_magnitude = 1.0;
8     double temperature = 1.0;
9
10    Główna pętla symulacji (wykonuj, dopóki t < max_iter ORAZ max_force_magnitude > min_force)
11    max_force_magnitude = 0.0;
12
13    Wyzeruj wszystkie siły dla bieżącej iteracji
14
15
16    // Oblicz siły odpychania między wszystkimi parami wierzchołków
17    Dla każdej pary wierzchołków
18        Obliczanie sił odchyłania
```

```

19
20
21
22 // Oblicz siły przyciągania wzdłuż krawędzi
23 Dla każdej krawędzi
24     Obliczanie sił przyciągania
25
26 // Zaktualizuj pozycje wierzchołków i znajdź maksymalną siłę
27 Dla każdego wierzchołka
28     Aktualizacja pozycji
29     Znajdź maksymalnej wartości siły
30
31     Zastosowanie siły do pozycji wierzchołka, skalując przez temperaturę
32
33 }
34
35 // Schładzanie
36 temperature *= cooling;
37 t++;
38

```

Główna funkcja sterująca procesem układania grafu. W każdej iteracji obliczane są siły odpychania (pomiędzy wszystkimi wierzchołkami) oraz siły przyciągania (tylko wzdłuż krawędzi). Pozycje wierzchołków są aktualizowane o wektor wypadkowej siły pomnożony przez aktualną „temperaturę” układu, która maleje w czasie, co pozwala na stabilizację rysunku.

Argumenty:

- graph - wskaźnik na strukturę grafu.
- minimum_force - próg siły, poniżej którego algorytm kończy pracę.
- max_iterations - limit iteracji pętli.
- ideal_len - docelowa długość krawędzi.
- spring_const - stała sprężystości (przyciąganie).
- c - stała potrzebna dla obliczenia siły odpychania.
- cooling - współczynnik chłodzenia układu (z zakresu 0 do 1).

4.7 Moduł toutes

Moduł ten zawiera implementację algorytmu Tutte’a (metoda barycentryczna), służącego do wyznaczania planarnego ułożenia grafu. Algorytm opiera się na unieruchomieniu wierzchołków poligonu zewnętrznego na planie wielokąta wypukłego i rozmieszczeniu pozostałych wierzchołków w środkach ciężkości ich sąsiadów.

4.7.1 Funkcja toutes_algorithm

Jest to główna funkcja sterująca procesem rozmieszczania grafu. Proces ten został podzielony na fazę przygotowawczą oraz iteracyjną pętlę obliczeniową.

Argumenty:

- graph - wskaźnik na strukturę grafu.
- max_iterations - maksymalna liczba kroków algorytmu.

Funkcja zwraca EXIT_SUCCESS w przypadku powodzenia lub kod błędu alokacji.

W pierwszej części funkcja alokuje pamięć na listy sąsiedztwa oraz pomocniczą tablicę new_pos, która przechowuje nowo obliczone współrzędne przed ich ostatecznym zapisaniem do struktury grafu.


```

1 // Budowanie listy sąsiedztwa
2 int result_adj_list = build_adj_list(graph, adj_list, deg);
3 Sprawdzanie budowania result_adj_list;
4
5
6
7 //3.=====Część obliczeniowa=====
8 // Przygotowanie tablicy wierzchołków poligonu zewnętrznego (fixed)
9
10 Zapełnienie is_fixed[]
11
12 // Ustawienie środka i rozmieszczenie punktów zewnętrznych na okręgu
13 Vector center = get_center(graph);
14 place_on_circle(dfs_res, graph, idx, center);
15

```

Następnie graf jest analizowany w celu znalezienia cyklu (ściany zewnętrznej). Wierzchołki należące do tego cyklu są oznaczane jako fixed (stałe) i rozmieszczane na okręgu za pomocą funkcji place_on_circle.

```

1
2
3 int t = 0;
4 max_change = 10.0;
5 /// 4. Główna pętla obliczeniowa
6 Główna pętla symulacji (wykonuj, dopóki t < max_iterations ORAZ max_change > MINIMUM_CHANGE):
7
8     t++;
9     max_change = 0.0;
10
11     //4.1.Liczmy idealne pozycje za pomocą algorytmu Tutte
12     Dla każdego wierzchołka
13         tutte_iteration(graph,i, is_fixed, adj_list,new_pos, deg, center);
14
15     //4.2.Itercja po każdej parze nowych koordynat dla wierzchołków, jeśli one mają tę samą pozycję, to
16     ↪ rozsuwamy je
17     Dla każdej pary wierzchołków
18         if(Jeśli mają tę samą pozycję ORAZ nie należą do poligonu zewnętrznego){
19             offset(new_pos, a,b, difference);
20
21     //4.3.Sprawdzenie warunku pętli while oraz zapis policzonych pozycji
22     Dla każdego wierzchołka
23         Jeśli nie należy do poligonu zewnętrznego
24             Liczymy różnicę między nową a starą pozycję i porównujemy z nową
25
26         Zapis końcowych pozycji do Graph
27
28
29     //Zwolnienie zaalokowanej pamięci
30     free_adj_list(graph, adj_list);
31     free_deg(deg);
32
33
34     return EXIT_SUCCESS;
35 }

```

Główna pętla realizuje iteracyjne przybliżanie pozycji wierzchołków do ich stanów równowagi. Dodatkowo funkcja sprawdza kolizje (nakładające się pozycje) i stosuje losowe przesunięcie (offset) w celu ich rozdzielenia. Pętla kończy się po osiągnięciu limitu iteracji lub gdy maksymalna zmiana pozycji wierzchołka spadnie poniżej progu MINIMUM_CHANGE.